# 01. 차원 축소(Dimension Reduction) 개요

## 다차원 피처의 문제점

- 데이터 포인트 간의 거리가 기하급수적으로 멀어져, sparse한 구조
- 상대적으로 적은 차원에서 학습된 모델보다 예측 신뢰도 하락
- 개별 피처 간의 높은 상관관계 우려

→ **차원 축소** : 매우 많은 다차원의 피처를 차원 축소해 새로운 차원의 데이터 세트를 생성하는 것. 3차원 이하의 차원축소를 하면, 시각적으로 표현도 가능해짐.

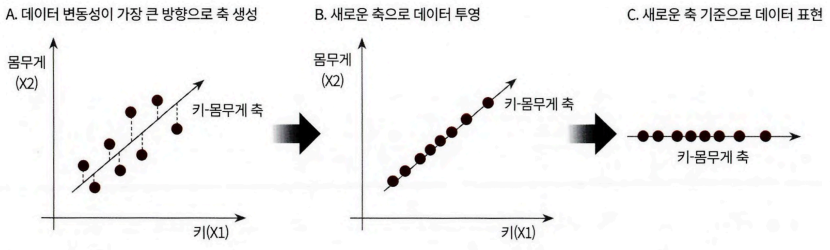
## 차원 축소의 종류

- 피처 선택 : 특정 피처의 종속성이 강한 불필요한 피처를 제거하고, 데이터의 특징을 잘 나타내는 주요 피처만 선택
- 피처 추출 : 기존 피처를 저차원의 중요 피처로 압축해서 추출.
  - ex. 학생의 모의고사 성적, 수능 성적, 대외활동 등 매우 많은 피처 -> 학업 성취도, 문제 해결력 등 데이터를 잘 설명할 수 있는 **잠재적 요소 추출**
  - **PCA, SVD, NMF는 잠재적 요소를 찾는 대표적인 차원 축소 알고리즘**

## 활용

- 이미지에서의 활용: 차원의 수가 너무 많은 원본 이미지는 적은 픽셀의 차이가 잘못된 예측으로 이어질 수 있음. 차원축소한 이미지는 이미지 분류 등의 분류 작업 수행 시 과적합 우려 적음.
- 텍스트에서의 활용 : 문서 내 단어들의 구성에서 숨겨져 있는 시맨틱 의미나 토픽을 잠재 요소로 간주하고 찾아낼 수 있음. SVD, NMF가 이에 사용됨.

# 02. PCA(Principal Component Analysis)




## 기본 원리

- 여러 변수간에 존재하는 상관관계를 이용해 이를 대표하는 주성분을 추출해 차원을 축소하는 기법. 기존 정보 유실 최소화가 목표임.
- **가장 높은 분산을 가지는 데이터 축을 찾아, 그 축(=PCA의 주성분)으로 데이터를 투영해 차원을 축소.**
  - 첫 번째 벡터축이자 주성분(PC1) : 데이터 분산이 가장 큰 방향
  - 두 번째 벡터축이자 주성분(PC2) : PC1과 직교하면서 그 다음으로 분산 큰 방향
  - n+1 번째 벡터축이자 주성분(PCn+1) : PCn과 직교하면서 그 다음으로 분산 큰 방향
  - → 벡터 축의 개수 = 차원.

## 선형대수 관점

$$C = P \sum P^T$$

공분산 행렬의 고유값 분해

$$C = [e_1 \cdots e_n] \begin{bmatrix} \lambda_1 & \cdots & 0 \\ \cdots & \cdots & \cdots \\ 0 & \cdots & \lambda_n \end{bmatrix} \begin{bmatrix} e_1^t \\ \cdots \\ e_n^t \end{bmatrix}$$

공분산 행렬의 고유값 분해

- **핵심: 입력 데이터의 공분산 행렬이 고유값 분해되어서, 고유벡터를 새로운 축으로 사용.**
- **C(공분산 행렬)= 고유벡터 orthogonal 행렬 * 고유값 정방행렬 * (고유벡터 orthogonal행렬)T**
  - 고유벡터 : 행렬 A를 곱해도 방향이 변하지 않고 크기만 변하는 벡터. Ax= ax(x: 고유벡터. a: 고유값). 정방행렬은 최대 그 차원수만큼의 고유벡터를 가질 수 있다. -> 새로운 축의 방향
  - 고유값: 해당 축 방향의 분산 크기
  - 공분산 행렬 : 개별 분산값을 대각 원소로 하는 대칭, 정방 행렬.
- PCA 구하는 step
  - 1. 입력 데이터 세트의 공분산 행렬 생성
  - 2. 공분산 행렬의 고유벡터와 고유값 계산
  - 3. 고유값이 가장 큰 순으로 K개(PCA 변환 차수만큼) 만큼 고유벡터를 추출
  - 4. 고유값이 큰 순으로 추출된 고유벡터를 이용해 새롭게 입력 데이터를 변환

### 사이킷런의 PCA 클래스

- PCA 는 여러 속성의 값을 연산해야하므로, 그 전에 StandardScaler로 (Target 속성을 제외한)속성의 스케일링이 필요.
- **객체= PCA(n_components= 변환할 차원의 수)**
  - **fit(입력 데이터세트), transform(입력 데이터세트)** 로 변환 수행. -> transform의 반환값=변환된 넘파이 행렬
- **explained_variance_ratio_** 속성 : 전체 변동성에서 개별 PCA 컴포넌트별로 차지하는 변동성 비율

## 03. LDA(Linear Discriminant Analysis, 선형 판별 분석)

- PCA와의 비교
  - **데이터 세트가 스케일링으로 데이터 중심이 동일해지면, 사이킷런의 PCA, SVD는 동일한 변환 수행. → PCA가 SCD 알고리즘으로 구현되었기 때문.**

| | PCA | LDA |
|---|---|---|
| 공통점 | 입력 데이터 세트를 저차원 공간에 투영해 차원 축소 | |
| | 이미지 압축을 통한 패턴 인식, 신호 처리 분야에 사용, 텍스트의 토픽 모델링 기법인 LSE의 기반 알고리즘. | |
| 차이점 | 입력 데이터의 변동성이 가장 큰 축을 찾음(비지도학습) | **입력 데이터의 결정 값 클래스를 최대한 분리할 수 있는 축을 찾음.(지도학습)** |
| | 공분산 행렬을 생성 | 클래스간 분산과 클래스 내부분산 행렬을 생성. |
| | 밀집 행렬(Dense Matrix)에 대한 변환만 가능. | 희소 행렬(Sparse Matrix)에 대한 변환도 가능. |

- **핵심 : 클래스간 분산은 최대, 클래스 내부의 분산 최소**
- LDA 구하는 step

- 1. 클래스 내부와 클래스 간 분산 행렬을 구함
- 2. 클래스 내부 분산 행렬을 Sw, 클래스 간 분산 행렬을 Sb라고 하면 다음 식으로 두 행렬을 고유벡터로 분해 가능.

$$S_W^T S_B = \begin{bmatrix} e_1 & \cdots & e_n \end{bmatrix} \begin{bmatrix} \lambda_1 & \cdots & 0 \\ \cdots & \cdots & \cdots \\ 0 & \cdots & \lambda_n \end{bmatrix} \begin{bmatrix} e_1^T \\ \cdots \\ e_n^T \end{bmatrix}$$

고유값 분해

- 3. 고유값이 가장 큰 순으로 K개(LDA 변환 차수만큼) 추출
- 4. 고유값이 가장 큰 순으로 추출된 고유벡터를 이용해 새롭게 입력 데이터를 변환

### 사이킷런의 LinearDiscriminantAnalysis 클래스

- PCA와 마찬가지로 StandardScaler로 (Target 속성을 제외한)속성의 스케일링이 필요.
- PCA와 달리, 지도학습이므로 결정값(iris.target)이 변환시 필요.
- **객체= LinearDiscriminantAnalysis(n_components= 변환할 차원의 수)**
  - **fit(입력 데이터세트, 결정값), transform(입력 데이터세트)** 로 변환 수행. -> transform의 반환값=변환된 넘파이 행렬

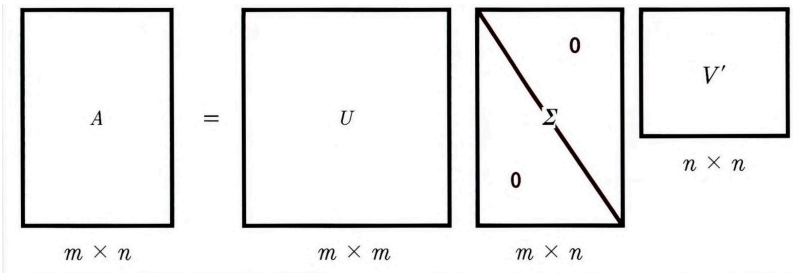
## 04. SVD(Singular Value Decomposition, 특이값 분해)

### SVD 개요

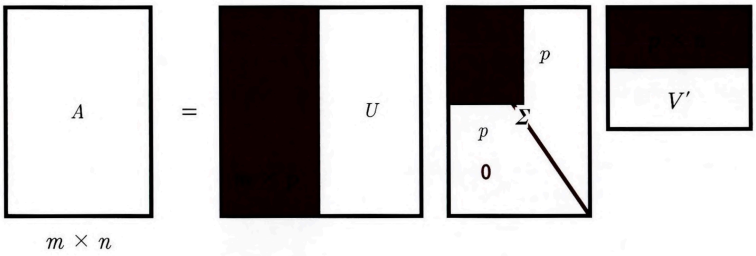
- PCA와의 차이점 : 정방행렬이 아닌 행렬에도 적용 가능.
- **SVD 기본 수식**
  - A(m*n): 원본데이터
  - ∑ (m*n): 대각행렬, 0이 아닌 값이 행렬A의 특이값
  - U(m*m), VT(n*n): 이 행렬에 속한 벡터들이 특이 벡터. 모든 특이 벡터는 서로 직교함
  - 사이파이의 svd : svd(원본행렬) → U, Sigma, Vt 행렬 반환

$$A = U \sum V^T$$

특이값 분해식




- **SVD 컴팩트한 형태** : 위의 ∑ 의 비대각인 부분과 대각 원소 중에서도 특이값이 0인 부분 제거.
  - U(m*p), ∑(p*p), VT(p*n)



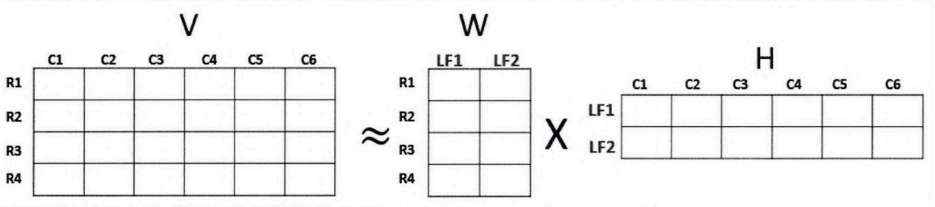

- **Truncated SVD** : ∑ 의 대각원소 중에 상위 몇 개만 추출해서 여기에 대응하는 U와 V의 원소도 함께 제거해 더욱 차원을 줄인 형태로 분해.
  - 더 작은 차원의 U, ∑, Vt로 분해하기 때문에 원본 행렬로 원복할 수 x. 원본 행렬로 근사만 가능.
  - 사이파이의 svds : svds(원본행렬, k= p값) → Truncated 형태의 U, Sigma, Vt 행렬 반환

## 사이킷런 TruncatedSVD 클래스를 이용한 변환

- **객체= TruncatedSVD(n_components= 변환할 차원의 수)**
  - **fit(입력 데이터세트), transform(입력 데이터세트)** 로 변환 수행. -> transform의 반환값=변환된 넘파이 행렬
- StandardScaler로 스케일링을 한 데이터러 PCA, SVD를 하면, 동일한 결과

# 05. NMF(Non-Negative Matrix Factorization)

- **원본 행렬을 두 개의 기반 양수 행렬로 분해.**
- Truncated SVD와 같이 낮은 랭크를 통한 행렬 근사 방식.
- 조건 : 원본 행렬의 모든 원소 값 >=0
- 행렬 분해
  - W(원본 행렬의 행 크기와 같고, 열 크기보다 작은 행렬) : 원본 행에 대해서 이 잠재 요소의 값이 얼마나 되는지 나타냄.
  - H(원본 행렬의 행 크기보다 작고 열 크기와 같은 행렬) : 잠재 요소가 원본 열(속성)로 어떻게 구성됐는지 나타냄.




- 활용 : SVD와 유사하게 이미지 변환 및 압축, 텍스트의 토픽 모델링, 문서 유사도 및 클러스터링, 영화 추천

### 사이킷런의 NMF 클래스

- **객체= NMF(n_components= 변환할 차원의 수)**
  - **fit(입력 데이터세트, 결정값), transform(입력 데이터세트)** 로 변환 수행. -> transform의 반환값=변환된 넘파이 행렬